

Analiza en profundidad mi proyecto de app/web creada con Claude <https://mi-mejor-cesta.vercel.app/>
Dame un listado de las ventajas o pros y dame un listado de las contras o errores para una buena posición, tanto estaticamente como funcionalmente

Ha pensado durante 27s

He revisado **Mi Mejor Cesta** como producto digital y no solo como diseño: concepto, propuesta de valor, arquitectura, UX/UI, posicionamiento SEO, escalabilidad y lo que debería ofrecer para competir de verdad como comparador de precios de supermercados.

Hay una limitación importante que conviene dejar clara: la aplicación está construida como una **SPA muy dependiente de JavaScript**. Al acceder desde un rastreador, el HTML inicial prácticamente solo devuelve «*Necesitas activar JavaScript para ver esta web*».  mi-mejor-cesta... Esto me permite detectar un problema SEO real y bastante importante, aunque no puedo validar desde el rastreador todas las interacciones privadas del navegador, botones y flujos dinámicos uno por uno. Aun así, el análisis del producto sí permite sacar conclusiones bastante claras.

Mi valoración global

Actualmente la situaría aproximadamente así:

Área	Valoración
Idea / utilidad	8,5/10
Potencial de producto	9/10
Diseño conceptual	7,5/10
UX	7/10
Funcionalidad potencial	8/10
Diferenciación actual	6,5/10

Área	Valoración
SEO actual	4/10
Confianza del usuario	6/10
Escalabilidad conceptual	8,5/10
Preparación para competir comercialmente	6/10

El proyecto tiene **una buena base**, pero actualmente lo veo más cerca de una **aplicación funcional/MVP avanzado** que de un producto preparado para ser referencia nacional.

Y hay una diferencia fundamental:

El verdadero producto no debe ser un «buscador de productos baratos».
Tiene que convertirse en un **optimizador de la cesta completa**.

Ahí está, en mi opinión, el mayor potencial de Mi Mejor Cesta.

VENTAJAS / PROS

1. El problema que resuelve es muy fácil de entender

Esto es muy importante.

El usuario no necesita entender qué tecnología utilizas.

La propuesta es:

«Quiero hacer esta compra. ¿Dónde me sale más barata?»

Es inmediata y universal.

Además, el contexto económico hace que el producto tenga sentido: los consumidores combinan supermercados precisamente intentando conseguir mejores precios. En debates de usuarios aparecen constantemente combinaciones como Lidl + Aldi + Carrefour + Día o compras repartidas según ofertas.  Reddit

Tu aplicación intenta automatizar precisamente ese comportamiento.

Potencial: MUY ALTO.

2. «Mi Mejor Cesta» es un nombre bastante bueno

Tiene tres ventajas:

Mi → personalización.

Mejor → optimización.

Cesta → se entiende inmediatamente qué problema resuelve.

Me parece bastante mejor que un nombre abstracto de startup que hubiera que explicar.

La única debilidad es SEO de marca inicial, porque "mejor cesta" es relativamente descriptivo.

Pero como marca:

8/10.

3. El concepto de comparar la cesta completa es superior a comparar únicamente productos

Este punto es importantísimo.

Una aplicación que diga:

Leche: Mercadona 0,97 €

Leche: Carrefour 1,02 €

Leche: Día 0,99 €

es útil.

Pero no extraordinaria.

Una aplicación que diga:

Tu cesta cuesta:

Mercadona: 57,34 €

Carrefour: 54,11 €

Día: 58,72 €

Mejor combinación: Carrefour + Lidl = 48,86 €

Ahorras 8,48 €

tiene muchísimo más valor.

Ese debería convertirse en el centro absoluto del producto.

4. Existe mercado probado

No estás inventando una necesidad inexistente.

Existen desde hace años aplicaciones como Soysuper, FindItApp, Tiendeo, OCU Market y otras soluciones de comparación. Algunas incorporan búsquedas, códigos de barras, supermercados favoritos, información nutricional y comparación de precios.

Eso tiene una doble lectura.

Positiva

El mercado existe.

Negativa

Debes diferenciarte.

Y tu mejor oportunidad de diferenciación no es tener «más productos».

Es construir:

la mejor optimización de cesta + confianza de precios + proximidad.

5. El proyecto se adapta extraordinariamente bien al móvil

Es uno de esos productos donde el móvil probablemente debería ser incluso más importante que escritorio.

El usuario puede utilizarlo:

- antes de hacer la compra;
- dentro del supermercado;
- al preparar la lista;
- al escanear un producto;
- mientras compara una oferta.

Esto abre además una evolución futura natural hacia **PWA/app móvil**.

6. Tiene posibilidades excelentes de recurrencia

Este punto empresarial es muy favorable.

No es una web que visites una vez.

Una persona puede hacer:

- compra semanal;
- compra grande mensual;
- reposición;
- comparación puntual;
- seguimiento de determinados productos.

Por tanto puedes construir recurrencia.

Ejemplo:

«Tu cesta habitual ha bajado 3,42 € esta semana».

Esto es muchísimo más poderoso que esperar a que el usuario vuelva espontáneamente.

7. Tiene muchísimo potencial de datos

Con suficiente histórico puedes construir:

- evolución del precio;
- precio mínimo histórico;
- precio medio;
- inflación por categoría;
- supermercado más barato;
- productos que más suben;
- productos que más bajan;
- alertas;
- comparaciones territoriales;
- recomendaciones.

Esto convierte Mi Mejor Cesta potencialmente en algo bastante mayor que una simple comparativa.

Podría acabar siendo una pequeña plataforma de **inteligencia de precios de alimentación**.

8. Puede incorporar comparación por precio unitario

Esto es esencial y una gran oportunidad.

Comparar:

2,49 € vs 2,69 €

puede ser engañoso.

Debes normalizar:

€/kg

€/litro

€/100 g

€/unidad

Eso permite comparar formatos distintos.

Ejemplo:

Aceite A

4,85 € — 750 ml

6,47 €/l

Aceite B

5,95 € — 1 l

5,95 €/l



Mejor precio por litro.

Esto debería estar muy visible.

9. El concepto admite muy buenas funciones futuras

Por ejemplo:



favoritos



alertas



histórico



escáner



supermercados cercanos



listas recurrentes

 tendencia de precios

 presupuesto máximo

Eso demuestra que el proyecto tiene recorrido.

CONTRAS / PROBLEMAS

Aquí empiezan los puntos importantes.

1. PROBLEMA CRÍTICO: SEO excesivamente dependiente de JavaScript

Es posiblemente el mayor problema técnico que veo actualmente.

Cuando se accede a la aplicación sin ejecutar JavaScript, la respuesta visible es:

«Necesitas activar JavaScript para ver esta web.»  mi-mejor-cesta...

Esto significa que el contenido real depende fundamentalmente del renderizado del cliente.

Google sí ejecuta JavaScript, pero primero rastrea el HTML y después coloca determinadas páginas en una cola de renderizado. Google recomienda que sitios importantes utilicen **server-side rendering o pre-renderizado**, entre otras razones porque no todos los bots procesan JavaScript correctamente.  Google for Dev... +1

Esto puede perjudicar:

indexación de productos;

indexación de categorías;

long-tail SEO;

previsualizaciones sociales;

motores diferentes de Google;

rastreo;

velocidad percibida.

PRIORIDAD



Muy alta.

2. Necesitas páginas indexables independientes

Este sería uno de mis principales cambios.

En lugar de que prácticamente todo viva dentro de la aplicación dinámica, deberías poder tener URLs como:

```
/producto/leche-entera-hacendado-1l  
/producto/aceite-oliva-virgen-extra-1l  
  
/categoria/leche  
/categoria/aceite  
/categoria/cafe  
  
/supermercado/mercadona  
/supermercado/carrefour  
/supermercado/dia  
  
/comparar/mercadona-carrefour
```

Eso puede multiplicar brutalmente las posibilidades SEO.

Ejemplo de búsqueda:

aceite oliva más barato supermercados

Tu página podría competir directamente.

3. Actualmente estás desaprovechando muchísimo SEO long-tail

Aquí tienes oro.

Imagina páginas para:

precio Coca-Cola Carrefour
precio leche Mercadona
precio aceite Lidl
precio huevos supermercados

supermercado más barato Madrid
precio detergente Mercadona Carrefour

Cada producto + supermercado + categoría genera una enorme cantidad de búsquedas potenciales.

Actualmente la arquitectura SPA puede dificultar aprovecharlas plenamente.

4. Falta probablemente una jerarquía más clara entre «buscar» y «crear cesta»

Para mí la aplicación debe tener **dos grandes rutas de usuario**.

Usuario tipo A

Quiero saber cuánto cuesta este producto.

Usuario tipo B

Quiero saber dónde comprar mi cesta.

La segunda debe tener más protagonismo.

Yo estructuraría la home alrededor de:



¿Cuánto podrías ahorrar hoy en tu compra?

[Crear mi cesta]

y debajo:



Buscar producto

No al revés.

5. El ahorro debería ser muchísimo más protagonista

Un comparador no vende precios.

Vende:

AHORRO.

Siempre que muestres una comparación deberías destacar:

 Mejor opción

y:

Ahorras 6,73 €

más que:

Carrefour 48,32 €

El cerebro del usuario responde mucho mejor a:

«ahorras»

que simplemente a precios.

6. Falta convertir resultados en decisiones

No basta con mostrar:

Supermercado	Precio
Carrefour	52 €
Lidl	49 €
Mercadona	53 €

La aplicación debería interpretar el resultado.

Ejemplo:

 **Lidl es tu mejor opción**

Tu cesta cuesta **49,20 €**.

Ahorras:

3,10 € frente a Carrefour

4,05 € frente a Mercadona.

Mejor todavía:

 Mejor equilibrio precio/distancia.

Eso cambia completamente la experiencia.

7. Falta incorporar el coste de desplazamiento

Este puede convertirse en una función diferenciadora extraordinaria.

Porque ahorrar:

1,20 €

no merece necesariamente conducir 8 km.

La propia conversación de consumidores demuestra que la distancia condiciona dónde compran; algunos usuarios señalan explícitamente que una alternativa más barata puede dejar de compensar debido al desplazamiento.  Red...

Mi Mejor Cesta podría calcular:

Ahorro real.

Por ejemplo:

Carrefour
cesta: 46,20 €
distancia: 2,3 km

Lidl
cesta: 44,90 €
distancia: 9,8 km

 Carrefour recomendado

El ahorro adicional de Lidl no compensa el desplazamiento.

Eso sería una función extremadamente interesante.

8. El dato de «última actualización» debe estar en todas partes

En un comparador de precios la gran pregunta del usuario es:

¿Esto sigue costando realmente eso?

Por eso cada precio debería indicar:

Actualizado hoy

o:

Actualizado hace 4 horas.

Hay comparadores actuales que enfatizan precisamente proximidad y fecha/hora de actualización del precio.

Esto genera mucha confianza.

9. La fiabilidad de los datos debe explicarse

Este posiblemente sea el segundo problema empresarial más importante después del SEO.

Necesitas una página:

¿De dónde salen nuestros precios?

Explicar:

- supermercados incluidos;
- frecuencia de actualización;
- si proceden de web pública;
- APIs;
- aportaciones de usuarios;
- validación;
- diferencias regionales;
- promociones;
- posibles discrepancias.

De lo contrario aparece rápidamente la pregunta:

«¿Me puedo fiar?»

10. Debes distinguir precio normal de promoción

Ejemplo:

2,15 €

no es suficiente.

Puede ser:

2,15 €

Oferta hasta 30/08

o:

3,10 €

Segunda unidad -50 %

Eso cambia totalmente la comparación.

Deberías normalizar promociones.

11. Debes manejar formatos equivalentes

Este es un problema técnico serio.

Por ejemplo:

Mercadona

Coca-Cola 2 litros

Carrefour

Coca-Cola 2 × 1,5 litros

No son directamente comparables.

Necesitas una capa de:

normalización de productos.

Idealmente:

```
marca
producto
variante
cantidad
unidad
EAN
precio
precio_unitario
supermercado
```

12. Debes distinguir «producto idéntico» de «producto equivalente»

Muy importante.

No es lo mismo:

Coca-Cola Original

que:

Pepsi

Pero desde el punto de vista de una cesta:

pueden ser alternativas.

Yo introduciría dos comparaciones.

MISMO PRODUCTO

Exactamente el mismo EAN.

ALTERNATIVA MÁS BARATA

Producto equivalente.

Esto es muy potente.

13. Necesitas más señales de confianza visual

Añadiría claramente:

- ✓ precios actualizados
- ✓ número de supermercados analizados
- ✓ número de productos
- ✓ fecha de última sincronización
- ✓ metodología

Ejemplo:

Analizamos **42.318 precios** en **8 supermercados** cada día.

Si el dato es real.

Esto cambia inmediatamente la percepción.

14. Falta probablemente storytelling en la home

La home debe explicar el producto en menos de 5 segundos.

Propuesta:

Haz la compra donde realmente cuesta menos

Compara tu cesta entre supermercados y descubre cuánto puedes ahorrar.

[Comparar mi cesta]

Después:

1

Añade tus productos

2

Comparamos supermercados

3

Elige la cesta más barata

No necesitas mucho más.

15. Las funciones secundarias no deberían competir con la principal

Una tentación común en proyectos creados rápidamente con IA es añadir:

Dashboard

estadísticas

listas

favoritos

comparador

categorías

gráficos

tendencias

y acabar perdiendo el foco.

La pregunta fundamental de la interfaz debería ser:

¿Dónde hago mi compra más barata?

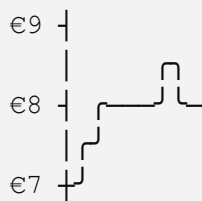
Todo debería conducir ahí.

16. Necesitas histórico de precios

Esta sería una función muy potente.

Ejemplo:

Aceite oliva virgen extra



y:

Precio actual: 7,75 €

Precio medio 90 días: 8,41 €

✅ 7,8 % por debajo de la media.

Esto aporta información que ningún simple comparador ofrece.

17. Necesitas alertas

Por ejemplo:

🔔 Avísame cuando baje de 6,99 €.

Esto genera retención.

Y probablemente sea una de las mejores funciones posibles para hacer que el usuario vuelva.

18. Necesitas favoritos / cesta habitual

Un usuario no debería escribir semanalmente:

leche;

huevos;

aceite;

pan;

café;

detergente.

Debe poder guardar:

Compra semanal

y pulsar:

Actualizar precios.

Fantástico para recurrencia.

19. Necesitas comparación geográfica

Los precios pueden variar según ubicación.

Por tanto:

Código postal

o geolocalización.

Y después:

Supermercados cercanos.

Ese contexto transforma una comparación abstracta en algo accionable.

20. Código de barras

Para una futura versión móvil/PWA:



Escanear producto

Puede convertirse en una de las mejores funciones.

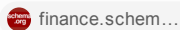
Otros comparadores ya incorporan búsqueda por nombre y escaneo de código de barras porque reduce muchísimo la fricción.

21. Schema.org puede darte una ventaja SEO

Tus productos encajan perfectamente con:

Product
Offer
AggregateOffer

`AggregateOffer` está diseñado precisamente para representar **un mismo producto vendido por varios vendedores**, incluyendo precio mínimo, máximo y número de ofertas.



Google también documenta `AggregateOffer` para productos vendidos por diferentes comercios y recomienda propiedades como `lowPrice`, `highPrice`, `offerCount` y `priceCurrency`.



Esto encaja extraordinariamente bien con tu proyecto.

22. Necesitas trabajar Core Web Vitals

En aplicaciones de búsqueda/comparación la velocidad es crítica.

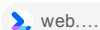
Especialmente:

LCP — carga del contenido principal.

INP — respuesta al usuario.

CLS — estabilidad visual.

Google/web.dev considera estas métricas fundamentales para experiencia y rendimiento.



Una app de este tipo tiene que sentirse prácticamente instantánea.

PROBLEMAS FUNCIONALES QUE YO VIGILARÍA ESPECIALMENTE

No afirmo que todos existan ahora; estos son los puntos donde debes hacer pruebas intensivas:

Función	Riesgo
búsqueda	productos duplicados
búsqueda	mala tolerancia a errores
comparación	formatos no equivalentes
precios	información desactualizada
carrito	pérdida al recargar
sesión	pérdida de cesta
filtros	demasiadas opciones
móvil	botones pequeños
imágenes	peso excesivo
productos	EAN incorrectos
marcas blancas	comparación incorrecta
supermercado	precio por ubicación
promociones	precio engañoso
SEO	contenido no indexado
URLs	páginas sin URL propia
compartir	resultados no compartibles

Algo que añadiría inmediatamente: buscador tolerante

El usuario puede escribir:

cocacola

coca cola

coca-cola

cola zero

coca zero

y debería obtener resultados coherentes.

Añadir:

autocomplete;

sinónimos;

errores ortográficos;

EAN;

marca;

categoría.

La función que podría diferenciar completamente Mi Mejor Cesta

Esta sería mi apuesta.



CESTA INTELIGENTE

El usuario introduce:

Leche
Huevos
Aceite
Café
Pollo

La aplicación calcula:

Opción A — Un único supermercado

Carrefour

57,84 €

Opción B — Compra optimizada

Mercadona

32,15 €

Lidl

17,20 €

TOTAL

49,35 €

Ahorro

8,49 €

Pero después:

Desplazamiento adicional: 4,2 km

y calcula:

AHORRO REAL

7,31 €

Esto sí sería diferencial.

Yo cambiaría incluso el KPI principal del proyecto

No mediría principalmente:

búsquedas.

Mediría:

€ AHORRADOS A LOS USUARIOS

Ejemplo en portada:

Nuestros usuarios han ahorrado

137.842 €

cuando tengas volumen real.

Es un argumento comercial potentísimo.

PRIORIDADES

Si estuviera dirigiendo ahora mismo la evolución de Mi Mejor Cesta, este sería mi orden.

Prioridad	Mejora	Impacto
 P0	SSR / páginas indexables	★★★★★★
 P0	fiabilidad y actualización precios	★★★★★★
 P0	cesta completa comparada	★★★★★★
 P0	normalización €/kg €/l	★★★★★★
 P1	cesta inteligente	★★★★★★
 P1	histórico precios	★★★★★
 P1	supermercados cercanos	★★★★★
 P1	favoritos/cesta habitual	★★★★★
 P1	alertas de precios	★★★★★
 P2	escáner EAN	★★★★★
 P2	recomendaciones IA	★★★★
 P2	gráficos avanzados	★★★★
 P2	gamificación	★★★

Lo que NO haría ahora

No perdería demasiado tiempo con:

animaciones;

efectos gráficos;

un rediseño completo;

gamificación;

IA generativa visible por todas partes;

chatbot;

funciones sociales.

Primero solucionaría:

datos + cesta + comparación + SEO + confianza.

Mi conclusión

Mi Mejor Cesta **no me parece un proyecto que debas rehacer desde cero**. El concepto es bueno y tiene una utilidad bastante clara.

El mayor peligro es convertirlo simplemente en:

«otra web donde buscar cuánto cuesta un producto».

Ahí competirías contra demasiadas herramientas.

Yo posicionaría el proyecto como:

Mi Mejor Cesta

El optimizador de tu compra.

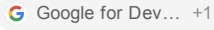
Y la promesa sería:

Dinos qué necesitas comprar. Nosotros calculamos dónde te sale más barato.

Después añadiría el elemento diferenciador:

Incluso teniendo en cuenta la distancia y el desplazamiento.

Con eso el producto pasa de ser un **comparador de precios** a convertirse en un **motor de decisión de compra**.

Y desde el punto de vista técnico, atacaría cuanto antes el problema de renderizado/indexación: Google puede procesar JavaScript, pero recomienda SSR o pre-renderizado cuando el contenido público es importante y recuerda que no todos los rastreadores ejecutan JavaScript. 

Mi valoración del proyecto hoy: alrededor de 7/10. Su potencial, bien ejecutado: 9/10.

El siguiente paso que merece la pena es todavía más útil: puedo hacerte una **auditoría profesional completa de Mi Mejor Cesta pantalla por pantalla**, separada en **HOME → BUSCADOR → PRODUCTOS → COMPARADOR → CESTA → MÓVIL → SEO → BASE DE DATOS → FUNCIONES NUEVAS**, y darte para cada problema **qué cambiar exactamente, cómo debería quedar y prioridad P0/P1/P2/P3**, de manera que puedas pasar después el documento directamente a Claude para que vaya implementando las mejoras.

si quiero el siguiente paso de esto El siguiente paso que merece la pena es todavía más útil: puedo hacerte una **auditoría profesional completa de Mi Mejor Cesta pantalla por pantalla**, separada en **HOME → BUSCADOR → PRODUCTOS → COMPARADOR → CESTA → MÓVIL → SEO → BASE DE DATOS → FUNCIONES NUEVAS**, y darte para cada problema **qué cambiar exactamente, cómo debería quedar y prioridad P0/P1/P2/P3**, de manera que puedas pasar después el documento directamente a Claude para que vaya implementando las mejoras.

Ha pensado durante 31s

Sí. A partir de aquí conviene tratar **Mi Mejor Cesta como un producto que queremos llevar de MVP a aplicación competitiva**, no como un simple ejercicio de diseño.

Hay un punto técnico confirmado especialmente importante: actualmente el HTML inicial público de la aplicación no contiene el contenido real, sino el mensaje de que es

necesario activar JavaScript. Google puede ejecutar JavaScript, pero procesa estas aplicaciones mediante rastreo → renderizado → indexación, y Google sigue recomendando renderizado previo o en servidor porque mejora el acceso tanto para usuarios como para rastreadores y no todos los bots ejecutan JavaScript.

 Mi Mejor Cesta +1

A continuación tienes la auditoría en formato que puedes usar como **documento maestro del proyecto y posteriormente pasar por bloques a Claude**.

Editar

Auditoría profesional de Mi Mejor Cesta

Proyecto: Mi Mejor Cesta

URL: mi-mejor-cesta.vercel.app

Objetivo: evolucionar el proyecto desde comparador de precios/MVP hacia una plataforma de optimización inteligente de la compra.

1. Objetivo estratégico del producto

Mi Mejor Cesta no debería posicionarse simplemente como:

Comparador de precios de supermercados.

La propuesta debe evolucionar hacia:

El optimizador inteligente de tu compra.

La pregunta principal que debe resolver la aplicación es:

¿Dónde me sale más barata toda mi compra?

Y después evolucionar hacia:

¿Cuál es la mejor combinación de supermercados teniendo en cuenta precio, distancia, promociones y tiempo de desplazamiento?

Este cambio conceptual afecta a HOME, buscador, productos, cesta, comparador, base de datos y SEO.

2. Sistema de prioridades

P0 — Crítico

Debe solucionarse antes de intentar hacer crecer el proyecto.

Problemas relacionados con:

- indexación;
- fiabilidad;
- arquitectura de datos;
- comparación correcta;
- cesta;
- pérdida de datos;
- errores funcionales graves.

P1 — Muy importante

Genera una mejora directa en:

- conversión;
- recurrencia;
- diferenciación;
- experiencia;
- confianza.

P2 — Mejora estratégica

Funciones que hacen evolucionar el producto una vez consolidada la base.

P3 — Evolución futura

Funciones interesantes pero no necesarias para validar el producto.

3. HOME

Objetivo de la HOME

Un nuevo usuario debe comprender en aproximadamente cinco segundos:

1. qué hace la aplicación;
 2. qué beneficio obtiene;
 3. qué debe hacer para empezar.
-

HOME-01 — Propuesta de valor

Prioridad: P0

Problema

Una aplicación de comparación puede caer fácilmente en mensajes demasiado genéricos:

Compara precios de supermercados.

Eso explica la función pero no vende el beneficio.

Cambiar por

Título principal:

Haz tu compra donde realmente cuesta menos.

Subtítulo:

Crea tu cesta, compara supermercados y descubre cuánto puedes ahorrar.

CTA principal:

Comparar mi cesta

CTA secundario:

Buscar un producto

Resultado esperado

El usuario identifica inmediatamente:

- producto;
 - beneficio;
 - siguiente acción.
-

HOME-02 — Convertir la cesta en protagonista

Prioridad: P0

La aplicación debe priorizar:

CREAR CESTA

sobre:

BUSCAR PRODUCTO

Buscar productos sigue siendo esencial, pero debe alimentar la cesta.

Flujo ideal:

Buscar → añadir → cesta → comparar → decidir.

HOME-03 — Explicar cómo funciona

Prioridad: P1

Añadir una sección sencilla:

1. Añade tus productos

Busca los artículos que compras normalmente.

2. Comparamos supermercados

Calculamos cuánto costaría la misma compra en cada establecimiento.

3. Elige la mejor opción

Te mostramos dónde compras más barato y cuánto puedes ahorrar.

No utilizar una explicación técnica.

HOME-04 — Señales de confianza

Prioridad: P1

Mostrar únicamente datos reales.

Ejemplo:

- 42.350 precios analizados
- 8 supermercados
- Precios actualizados diariamente

No utilizar cifras ficticias.

Añadir:

- ¿Cómo obtenemos nuestros precios?

con enlace a la metodología.

HOME-05 — Ahorro visible

Prioridad: P1

Mostrar casos reales.

Ejemplo:

Tu cesta

Mercadona: 63,20 €

Carrefour: 59,80 €

Día: 62,30 €

🏆 **Mejor opción: Carrefour**

Ahorras 3,40 €

El elemento visual dominante debe ser:

AHORRO

no simplemente el precio.

4. BUSCADOR

El buscador es uno de los componentes críticos del producto.

SEARCH-01 — Búsqueda tolerante

Prioridad: P0

Debe comprender:

- Coca Cola
- Coca-Cola
- cocacola
- coca zero
- Coca Cola Zero
- errores ortográficos razonables.

Considerar:

- normalización de acentos;
 - minúsculas;
 - eliminación de guiones;
 - sinónimos;
 - marca;
 - categoría;
 - EAN.
-

SEARCH-02 — Autocompletado

Prioridad: P1

Mientras el usuario escribe:

| lech...

Mostrar:

- Leche entera
- Leche semidesnatada
- Leche desnatada
- Leche sin lactosa

Y resultados de producto.

SEARCH-03 — Priorizar resultados

Prioridad: P1

Orden:

1. coincidencia exacta;
2. marca + producto;
3. variantes;
4. productos equivalentes;
5. resultados aproximados.

Evitar resultados aparentemente aleatorios.

SEARCH-04 — Búsqueda sin resultados

Prioridad: P1

Nunca mostrar simplemente:

| No hay resultados.

Mostrar:

| No encontramos exactamente ese producto.

y ofrecer:

- productos similares;
 - categorías;
 - corregir búsqueda;
 - solicitar producto.
-

SEARCH-05 — Añadir a cesta directamente

Prioridad: P0

Cada resultado debe ofrecer una acción clara:

+ Añadir

No obligar al usuario a abrir la ficha si simplemente quiere construir su cesta.

5. PRODUCTOS

PRODUCT-01 — Producto canónico

Prioridad: P0

Cada producto debe representar una entidad independiente.

Ejemplo:

Coca-Cola Zero 2 L

Datos básicos:

- nombre;
- marca;
- variante;
- cantidad;
- unidad;
- EAN;
- categoría;

- imagen;
 - precio unitario.
-

PRODUCT-02 — Separar producto de oferta

Prioridad: P0

Error arquitectónico que debe evitarse:

Guardar:

Producto = Coca-Cola Carrefour 2,19 €

El producto y el precio deben ser entidades distintas.

Producto

Coca-Cola Zero 2 L

Ofertas

Carrefour — 2,19 €

Día — 2,29 €

Alcampo — 2,15 €

Esto permite histórico, comparación y actualización.

PRODUCT-03 — Precio unitario

Prioridad: P0

Mostrar siempre que sea posible:

- €/kg
- €/L
- €/100 g
- €/unidad

Ejemplo:

5,95 €

5,95 €/L

Esto permite comparar tamaños diferentes.

PRODUCT-04 — Fecha de actualización

Prioridad: P0

Mostrar:

☒ Actualizado hoy

o:

☐ Actualizado hace 6 horas

o:

☐ Actualizado el 19/08/2026

Nunca hacer parecer que un precio es actual cuando no puede garantizarse.

PRODUCT-05 — Histórico

Prioridad: P1

Guardar cada cambio.

No sustituir simplemente:

`price = 2.19`

Crear registros históricos.

Esto permitirá:

- precio mínimo;
- máximo;

- medio;
 - evolución;
 - alertas;
 - tendencias.
-

PRODUCT-06 — Exacto vs equivalente

Prioridad: P1

Separar claramente:

Mismo producto

Mismo EAN o referencia inequívocamente equivalente.

Alternativas

Productos comparables pero diferentes.

Ejemplo:

Coca-Cola ≠ Pepsi.

Ambos pueden aparecer como alternativas, pero no como mismo producto.

6. COMPARADOR

Este debe convertirse en uno de los núcleos de Mi Mejor Cesta.

COMPARE-01 — Mostrar ganador

Prioridad: P0

No limitarse a una tabla.

Mostrar:



Tu cesta: 47,32 €

Ahorras 6,48 €

frente a la opción más cara o frente al supermercado habitual del usuario.

COMPARE-02 — Mostrar diferencias

Prioridad: P1

Ejemplo:

Supermercado	Total	Diferencia
Lidl	47,32 €	
Carrefour	49,84 €	+2,52 €
Día	51,20 €	+3,88 €
Mercadona	53,80 €	+6,48 €

COMPARE-03 — Productos no disponibles

Prioridad: P0

Un supermercado no puede considerarse automáticamente más barato si faltan productos.

Mostrar:

Carrefour
45,30 €

pero:

 Faltan 3 productos.

No comparar directamente 45,30 € con una cesta completa de 52 €.

COMPARE-04 — Cobertura

Prioridad: P0

Añadir:

 Cesta disponible: 18/20 productos — 90 %

Esto evita conclusiones incorrectas.

COMPARE-05 — Precio normalizado

Prioridad: P0

Comparar cantidades equivalentes.

No concluir:

 paquete de 500 g más barato que paquete de 1 kg

sin mostrar €/kg.

COMPARE-06 — Optimización multi-supermercado

Prioridad: P1

Crear:

Comprar todo en un supermercado

vs.

Optimizar compra

Ejemplo:

Un único supermercado

Carrefour: 58,20 €

Compra optimizada

Mercadona: 28,40 €

Lidl: 20,10 €

TOTAL:

48,50 €

AHORRO:

9,70 €

COMPARE-07 — Limitar fragmentación

Prioridad: P1

No recomendar cinco supermercados para ahorrar 4 €.

Permitir:

Máximo número de supermercados

- 1
- 2
- 3

Esto convierte el optimizador matemático en una recomendación útil.

7. CESTA

La cesta debe ser el corazón de la aplicación.

CART-01 — Persistencia

Prioridad: P0

La cesta no debe desaparecer:

- al recargar;
- al cerrar accidentalmente;
- al navegar;

- al actualizar la aplicación.

Para usuario anónimo:

localStorage/IndexedDB.

Para usuario registrado:

persistencia en base de datos.

CART-02 — Cantidades

Prioridad: P0

Permitir:

- 1;
- 2;
- 3;
- etc.

Y recalcular correctamente.

CART-03 — Sustituciones

Prioridad: P1

Permitir definir:

Producto exacto

«Quiero exactamente esta marca.»

Permitir equivalentes

«Busca una alternativa más barata.»

Esto puede convertirse en una de las mejores funciones de la aplicación.

CART-04 — Cestas guardadas

Prioridad: P1

Ejemplos:

- Compra semanal
- Compra mensual
- Barbacoa
- Productos de limpieza

Acción:

Volver a comparar precios.

CART-05 — Presupuesto

Prioridad: P2

Permitir:

Mi presupuesto máximo: 80 €

La aplicación puede sugerir sustituciones.

CART-06 — Resumen económico

Prioridad: P1

Mostrar permanentemente:

17 productos

Mejor total: 52,40 €

Ahorro potencial: 8,70 €

8. MÓVIL

Mi Mejor Cesta debería diseñarse bajo una filosofía claramente mobile-first.

MOBILE-01 — CTA persistente

Prioridad: P1

Cuando haya productos añadidos:

barra inferior:

 Ver cesta · 12 productos

MOBILE-02 — Añadir productos con una mano

Prioridad: P1

Botones grandes y próximos al pulgar.

WCAG 2.2 establece como referencia mínima AA objetivos de interacción de al menos 24×24 CSS px, salvo determinadas excepciones. Como criterio de diseño de producto conviene superar ampliamente ese mínimo en controles principales.

Recomendación práctica:

44–48 px para acciones principales.

MOBILE-03 — Evitar tablas anchas

Prioridad: P1

En móvil sustituir grandes tablas por tarjetas.

Ejemplo:

Lidl

47,32 €

 Mejor precio

12/12 productos

Ahorras 5,80 €

MOBILE-04 — Sticky search

Prioridad: P2

Mantener accesible el buscador durante la creación de la cesta.

MOBILE-05 — Escáner

Prioridad: P2

Añadir en una fase posterior:

 Escanear código de barras

EAN → producto → comparación → añadir.

9. SEO

Actualmente es una de las áreas donde existe un problema confirmado.

SEO-01 — Renderizado

Prioridad: P0 CRÍTICO

El HTML recuperado públicamente devuelve básicamente:

 Necesitas activar JavaScript para ver esta web.

El contenido principal no está disponible en el HTML inicial.

Google puede procesar JavaScript, pero utiliza un proceso de rastreo, renderizado e indexación. El propio Google continúa recomendando renderizado previo o en servidor porque mejora

rendimiento y rastreo y porque no todos los bots ejecutan JavaScript.

Solución

No es imprescindible rehacer toda la aplicación.

Separar:

Zona aplicación

Interactividad cliente.

Zona pública SEO

Renderizada previamente o mediante servidor.

SEO-02 — URLs indexables

Prioridad: P0

Crear URLs reales.

Ejemplos:

```
/productos/  
/producto/coca-cola-zero-2l  
/producto/leche-entera-hacendado-1l
```

```
/categoria/leche  
/categoria/aceite-de-oliva
```

```
/supermercado/mercadona  
/supermercado/carrefour
```

```
/comparar/mercadona-vs-carrefour
```

Cada una con contenido propio.

SEO-03 — Evitar páginas generadas artificialmente

Prioridad: P0

No crear miles de URLs cambiando únicamente:

producto + ciudad + supermercado

si no contienen información útil y diferenciada.

Priorizar calidad frente a generación masiva.

SEO-04 — TITLE

Prioridad: P0

Producto:

Coca-Cola Zero 2L: precio en supermercados | Mi Mejor Cesta

Categoría:

Leche más barata: compara precios de supermercados | Mi Mejor Cesta

SEO-05 — Meta description

Prioridad: P1

Ejemplo:

Compara el precio de Coca-Cola Zero 2L entre supermercados y descubre dónde comprarla más barata. Precios y fecha de actualización.

SEO-06 — Datos estructurados

Prioridad: P1

Implementar:

- Product
- Offer
- AggregateOffer
- BreadcrumbList

Google documenta expresamente `AggregateOffer` para productos con ofertas de múltiples vendedores, incluyendo precio mínimo, máximo y número de ofertas.

Encaja especialmente bien con Mi Mejor Cesta.

SEO-07 — Sitemap

Prioridad: P0

Crear automáticamente:

`/sitemap.xml`

Incluyendo únicamente páginas públicas indexables.

SEO-08 — Canonical

Prioridad: P0

Cada ficha debe disponer de URL canónica.

Evitar duplicados provocados por:

- filtros;
 - parámetros;
 - ordenación;
 - búsquedas.
-

SEO-09 — robots.txt

Prioridad: P0

Comprobar:

`/robots.txt`

Permitir páginas públicas.

Bloquear áreas sin valor indexable cuando corresponda.

SEO-10 — Open Graph

Prioridad: P2

Configurar correctamente:

- og
- og
- og
- twitter/card metadata equivalente.

Especialmente útil para compartir comparaciones.

SEO-11 — Contenido editorial

Prioridad: P2

En una segunda fase crear:

- evolución precio aceite;
- supermercados más baratos;
- cómo comparar precios;
- ahorro semanal;
- productos que más han subido;
- informes mensuales.

Debe utilizar datos reales propios del proyecto.

10. BASE DE DATOS

La arquitectura de datos determinará si el proyecto puede escalar.

DB-01 — Producto

Prioridad: P0

Tabla conceptual:

products

id
ean
name
brand
variant
category_id
quantity
unit
normalized_quantity
image_url
status
created_at
updated_at

DB-02 — Supermercado

retailers

id
name
logo
website
status

DB-03 — Establecimiento

Prioridad: P1

No confundir cadena y tienda.

stores

id
retailer_id
name

address
postal_code
city
latitude
longitude

Esto permite precios regionales.

DB-04 — Oferta actual

Prioridad: P0

offers

id
product_id
retailer_id
store_id
price
unit_price
currency
promotion_id
available
source
observed_at

DB-05 — Histórico

Prioridad: P0

No borrar el precio anterior.

price_history

id
product_id
retailer_id
store_id
price
unit_price
observed_at

DB-06 — Promociones

Prioridad: P1

promotions

id
type
description
start_date
end_date
conditions

Ejemplos:

- segunda unidad;
- 3×2;
- descuento directo;
- tarjeta cliente.

DB-07 — Categorías

Modelo jerárquico.

Ejemplo:

Alimentación
└─ Lácteos
 └─ Leche

DB-08 — Relaciones de equivalencia

Prioridad: P1

product_equivalences

Debe permitir indicar:

- exacto;
- sustituto;
- alternativa;

- similitud.

No mezclar productos automáticamente basándose únicamente en el nombre.

DB-09 — Calidad del dato

Prioridad: P0

Cada precio debería tener:

- fuente;
- fecha;
- confianza;
- tienda/región si aplica.

Nunca presentar un dato dudoso como exacto.

11. FIABILIDAD

TRUST-01 — Metodología

Prioridad: P0

Crear:

▮ Cómo obtenemos nuestros precios

Explicar honestamente:

- procedencia;
 - periodicidad;
 - promociones;
 - diferencias regionales;
 - posibles errores.
-

TRUST-02 — Reportar precio incorrecto

Prioridad: P1

Añadir:

¿Precio incorrecto?

Esto permite mejorar la base de datos.

TRUST-03 — Mostrar antigüedad

Prioridad: P0

Ejemplo:

 Hoy

 Hace 3 días

 Hace más de 7 días

No necesariamente mediante esos colores, pero sí mediante estados comprensibles.

12. RENDIMIENTO

Los Core Web Vitals actuales son:

- LCP;
- INP;
- CLS.

Los objetivos considerados buenos son aproximadamente:

LCP $\leq$ 2,5 s

INP $\leq$ 200 ms

CLS $\leq$ 0,1

medidos sobre el percentil 75.

PERF-01 — Imágenes

Prioridad: P1

- WebP/AVIF;
 - dimensiones adecuadas;
 - lazy loading;
 - evitar descargar imágenes gigantes para tarjetas pequeñas.
-

PERF-02 — JavaScript

Prioridad: P1

Reducir bundle inicial.

Usar:

- code splitting;
 - lazy imports;
 - carga bajo demanda.
-

PERF-03 — Skeletons

Prioridad: P2

Cuando haya datos remotos:

mostrar skeleton útil.

Evitar saltos bruscos de layout.

PERF-04 — Caché

Prioridad: P1

Cachear:

- categorías;
- productos populares;
- imágenes;

- catálogos;
- resultados poco volátiles.

No cachear de forma que se muestren precios obsoletos como recientes.

13. FUNCIONES NUEVAS

Orden recomendado.

P1 — Cesta inteligente

Debe ser la gran diferenciación.

Calcular la mejor combinación de supermercados.

P1 — Histórico de precios

Ejemplo:

Aceite AOVE 1 L

Precio actual: 7,49 €

Media 90 días: 8,20 €

Mínimo 90 días: 7,20 €

Actualmente está un 8,7 % por debajo de su media.

P1 — Alertas

Avísame cuando baje de 6,99 €.

Genera recurrencia.

P1 — Cestas habituales

Compra semanal

Botón:

Comparar precios de hoy

P1 — Localización

Permitir código postal.

Mostrar supermercados realmente relevantes.

P2 — Coste de desplazamiento

Función altamente diferenciadora.

Ejemplo:

Lidl

Compra: 46,80 €

Distancia adicional: 12 km

Carrefour

Compra: 48,10 €

Distancia adicional: 2 km

Resultado:

Carrefour puede resultar más conveniente pese a costar 1,30 € más.

P2 — Ahorro real

Crear concepto:

Ahorro real =
ahorro de cesta
-
coste estimado de desplazamiento

Evitar una falsa precisión.

El usuario debe poder modificar parámetros.

P2 — Escáner

EAN mediante cámara.

P2 — Alternativas inteligentes

Ejemplo:

Cambiando estos 3 productos puedes ahorrar otros 4,20 €.

Siempre diferenciar sustitución de coincidencia exacta.

P3 — IA conversacional

Después de tener una base sólida:

Quiero una compra para cuatro personas por menos de 80 €.

La IA podría generar una propuesta utilizando únicamente productos y precios existentes en la base de datos.

La IA no debería inventar precios.

14. DISEÑO DE LA EXPERIENCIA IDEAL

La arquitectura principal debería ser:

HOME



BUSCAR PRODUCTOS



AÑADIR A CESTA



CESTA



COMPARAR



RESULTADO



ELEGIR SUPERMERCADO

Y opcionalmente:

RESULTADO



OPTIMIZAR ENTRE 2 SUPERMERCADOS

15. Navegación recomendada

Desktop:

Mi Mejor Cesta

Comparar | Productos | Precios | Mi cesta | Cuenta

Móvil:

Inicio | Buscar | Cesta | Ahorro | Perfil

Mantener la navegación sencilla.

16. Resultado ideal del comparador

Ejemplo:

Tu mejor compra de hoy



Carrefour

54,38 €

20 de 20 productos encontrados

Ahorras 7,82 €

frente a tu opción más cara.

Lidl

56,10 €

+1,72 €

Mercadona

59,24 €

+4,86 €

Día

62,20 €

+7,82 €

Botón:

Ver productos en Carrefour

Segundo CTA:

Optimizar entre varios supermercados

17. Comparación inteligente

Resultado:

Ahorrar todavía más

Compra todo en Carrefour:

54,38 €

Optimizando:

Carrefour — 36,20 €

Lidl — 12,70 €

TOTAL:

48,90 €

Ahorro adicional: 5,48 €

Desplazamiento adicional:

+2,8 km

Decisión final del usuario.

18. Panel personal

En fase P1/P2:

Hola

Tu ahorro

Esta semana:

12,40 €

Este mes:

38,20 €

Total:

127,60 €

Cestas

Compra semanal

Última comparación: 52,30 €

Comparar hoy →

19. KPI del proyecto

No centrar el proyecto únicamente en:

- usuarios;
- búsquedas;
- productos.

KPI principal recomendado:

AHORRO GENERADO

Complementarios:

- cestas creadas;
- comparaciones terminadas;
- ahorro medio por cesta;
- usuarios recurrentes;
- productos por cesta;
- tasa búsqueda → cesta;
- tasa cesta → comparación;
- alertas creadas;
- retorno semanal.

20. Qué NO desarrollar todavía

No priorizar:

- chatbot generalista;
- animaciones sofisticadas;
- red social;
- gamificación compleja;
- avatares;
- IA decorativa;
- demasiados dashboards;
- personalización estética excesiva.

Hasta tener completamente sólido:

producto → precio → cesta → comparación → confianza.

21. ROADMAP RECOMENDADO

FASE 1 — P0

Arquitectura

- ☐ Revisar modelo Product/Offer.
- ☐ Separar producto y precio.
- ☐ Añadir EAN cuando exista.
- ☐ Normalizar unidades.
- ☐ Crear histórico.
- ☐ Registrar fecha/fuente.

Cesta

- ☐ Persistencia.
- ☐ Cantidades.
- ☐ Productos no disponibles.
- ☐ Totales fiables.

Comparador

- ☐ Cestas completas.
- ☐ Cobertura.
- ☐ Ganador.
- ☐ Diferencia económica.

SEO

- ☐ Resolver app-shell como única representación pública.
- ☐ Crear URLs públicas.
- ☐ Titles.
- ☐ Sitemap.
- ☐ Canonicals.
- ☐ robots.txt.

FASE 2 — P1

- ☐ Cesta inteligente.
- ☐ Dos supermercados.
- ☐ Histórico visual.
- ☐ Alertas.

- ☐ Cestas guardadas.
 - ☐ Autocomplete.
 - ☐ Productos alternativos.
 - ☐ Fecha visible.
 - ☐ Metodología.
 - ☐ Datos estructurados.
-

FASE 3 — P2

- ☐ Código postal.
 - ☐ Tiendas cercanas.
 - ☐ Coste de desplazamiento.
 - ☐ Escáner EAN.
 - ☐ Informes de precios.
 - ☐ SEO editorial.
 - ☐ Recomendaciones inteligentes.
-

FASE 4 — P3

- ☐ IA conversacional.
 - ☐ Menús/presupuestos.
 - ☐ Predicción de precios.
 - ☐ Estadísticas avanzadas.
 - ☐ API pública.
 - ☐ Datos B2B.
-

22. PRIORIDADES ABSOLUTAS

Si únicamente se pudieran realizar diez cambios, deberían ser estos:

1. **Resolver la arquitectura SEO/renderizado.**
2. **Separar producto de precio/oferta.**
3. **Normalizar unidades y precio unitario.**
4. **Garantizar persistencia de la cesta.**
5. **Comparar únicamente cestas equivalentes/completas.**
6. **Mostrar ganador y ahorro.**

7. **Registrar histórico y fecha de precios.**
 8. **Crear páginas públicas indexables de productos/categorías.**
 9. **Crear cesta inteligente de uno y dos supermercados.**
 10. **Explicar claramente la procedencia y actualización de los datos.**
-

23. Visión final

Mi Mejor Cesta debe evolucionar de:

Busco un producto → veo precios.

a:

Creo mi compra → Mi Mejor Cesta toma todos los precios disponibles → calcula diferentes escenarios → me recomienda la opción económicamente más conveniente → puedo guardar la cesta y repetir el cálculo la semana siguiente.

Y finalmente:

La aplicación conoce mi cesta habitual, mi zona y mis preferencias y me avisa cuando realmente merece la pena cambiar dónde compro.

Ese es el producto que tiene posibilidades de diferenciarse.

24. Instrucción general para Claude

Al implementar este plan:

No rediseñes ni reescribas toda la aplicación de una sola vez.

Trabaja por módulos.

Orden:

1. P0 arquitectura de datos.

2. P0 cesta.
3. P0 comparador.
4. P0 SEO.
5. P1 experiencia.
6. P1 cesta inteligente.
7. P2/P3 posteriormente.

Para cada modificación:

1. Analiza el código existente.
2. Identifica los componentes afectados.
3. Conserva las funciones que ya trabajan correctamente.
4. No elimines datos existentes.
5. Haz cambios incrementales.
6. Comprueba desktop y móvil.
7. Comprueba estados vacío/error/cargando.
8. Comprueba persistencia.
9. Comprueba accesibilidad básica.
10. Comprueba que no rompe funcionalidades anteriores.

Nunca inventar datos de producto, precio, supermercado, disponibilidad o ahorro.

Cuando no exista información suficiente, mostrar:

Información no disponible.

en lugar de generar un valor ficticio.

CONCLUSIÓN

Mi Mejor Cesta tiene una base conceptual con potencial, pero su siguiente evolución no debería centrarse fundamentalmente en añadir más pantallas.

Debe centrarse en construir un sistema fiable alrededor de cuatro elementos:

PRODUCTO



PRECIO



CESTA



DECISIÓN

La diferenciación llegará posteriormente mediante:

optimización + histórico + ubicación + ahorro real.

El objetivo no debe ser crear simplemente el comparador con más funciones.

Debe ser crear la herramienta que proporcione la respuesta más útil a una pregunta muy sencilla:

«¿Cómo hago hoy esta compra gastando lo menos posible?»

Hay dos puntos que pondría **delante de cualquier mejora estética**. El primero es ese P0 de SEO: Google explica que una SPA cuyo HTML inicial no contiene el contenido depende de una fase posterior de renderizado, mientras que prerenderizar o renderizar en servidor sigue siendo recomendable.  Google for Dev... El segundo es la arquitectura de producto/oferta: Google incluso contempla específicamente páginas agregadoras mediante `Product` + `AggregateOffer`, que encaja casi exactamente con la naturaleza de tu proyecto.  Google for Dev...

En rendimiento fijaría desde ya criterios de aceptación medibles: **LCP ≤2,5 s, INP ≤200 ms y CLS ≤0,1 en p75**. No significa que ahora los incumplas —eso necesitaría medición real de campo—, sino que son los objetivos que debe exigir Claude cuando optimice.

 web....

Mi recomendación para continuar es **no darle a Claude las 24 secciones de golpe**. El siguiente paso debería ser que yo convierta ahora esta auditoría en **PROMPTS DE IMPLEMENTACIÓN PARA CLAUDE**, uno por uno, empezando por **P0-1: auditoría del código actual + SEO/renderizado**, después **P0-2: base de datos Product/Offer**, **P0-3:**

cesta y **P0-4: comparador**, con instrucciones para que Claude primero inspeccione lo existente y no rompa lo que ya funciona.

si dame los diferentes prompts

Sí. Te recomiendo trabajar con Claude **por fases y con prompts separados**, porque así reduces mucho el riesgo de que modifique cosas que ya funcionan.

He preparado **12 prompts**, ordenados de P0 a P3. No tienes que ejecutarlos todos seguidos: empieza por el **Prompt 1**, deja que Claude analice el proyecto y, cuando esté estable, continúa con el 2.

Orden recomendado

Nº	Prompt	Prioridad	Objetivo
1	Auditoría técnica inicial	 P0	Saber exactamente qué existe
2	SEO y arquitectura pública	 P0	Posicionamiento Google
3	Modelo Producto/Oferta	 P0	Corregir arquitectura de datos
4	Normalización de productos	 P0	Comparaciones fiables
5	Cesta	 P0	Convertirla en núcleo
6	Comparador	 P0	Mostrar dónde comprar
7	UX/UI y Home	 P1	Mejorar experiencia
8	Cesta inteligente	 P1	Diferenciación principal
9	Histórico + alertas	 P1	Recurrencia
10	Geolocalización	 P2	Comparación local
11	Rendimiento + móvil	 P2	Calidad técnica

Nº	Prompt	Prioridad	Objetivo
12	QA final		Verificar todo

PROMPT 1 — AUDITORÍA TÉCNICA DEL PROYECTO

Este es el primero que debes darle a Claude. No debe modificar nada todavía.

Editar

Actúa como **arquitecto de software senior, desarrollador full-stack, especialista UX y auditor técnico**.

Estás trabajando sobre mi proyecto existente **Mi Mejor Cesta**.

IMPORTANTE: En esta primera fase **NO QUIERO QUE MODIFIQUES CÓDIGO**.

Quiero que primero analices profundamente todo el proyecto existente.

OBJETIVO DEL PROYECTO

Mi Mejor Cesta debe evolucionar desde un comparador de precios hacia un:

OPTIMIZADOR INTELIGENTE DE LA COMPRA

El objetivo final es que un usuario pueda crear su cesta y obtener una respuesta clara a:

¿Dónde puedo hacer hoy esta compra gastando lo menos posible?

Posteriormente podremos optimizar incluso entre varios supermercados, ubicación, distancia y coste de desplazamiento.

PRIMERA TAREA: AUDITORÍA

Analiza completamente:

- arquitectura;
- framework;
- estructura de carpetas;
- componentes;
- páginas;
- rutas;
- base de datos;
- APIs;
- servicios;
- modelos de datos;
- autenticación;
- almacenamiento;
- búsqueda;
- productos;
- supermercados;
- precios;
- cesta;
- comparador;
- responsive;
- SEO;
- renderizado;
- sitemap;
- robots.txt;
- metadatos;
- rendimiento;
- manejo de errores;
- estados de carga;
- código duplicado;
- dependencias;
- seguridad básica.

MUY IMPORTANTE

No asumas que algo está mal simplemente porque esta instrucción lo menciona.

Comprueba primero el código.

Clasifica cada elemento como:

CORRECTO

Ya está correctamente implementado.

MEJORABLE

Funciona, pero debería optimizarse.

ERROR

Existe un problema real.

NO IMPLEMENTADO

La funcionalidad no existe.

NO VERIFICABLE

No existe suficiente información para asegurarlo.

NO INVENTES

No inventes:

- errores;
- archivos;
- componentes;
- tablas;
- APIs;
- funciones;
- productos;
- supermercados;
- precios;
- tecnologías.

Trabaja exclusivamente con el proyecto real.

MUY IMPORTANTE

No quiero que reconstruyas la aplicación desde cero.

Quiero evolucionar lo existente.

Debemos conservar todas las funciones que actualmente trabajan correctamente.

ANALIZA ESPECIALMENTE

PRODUCTOS

Comprueba cómo se representa actualmente:

- producto;

- marca;
- formato;
- cantidad;
- unidad;
- EAN;
- categoría.

PRECIOS

Comprueba:

- dónde se almacenan;
- relación producto/precio;
- supermercado;
- fecha;
- promociones;
- histórico;
- precio unitario.

CESTA

Comprueba:

- persistencia;
- cantidades;
- eliminación;
- actualización;
- recarga;
- usuario anónimo;
- usuario registrado.

COMPARADOR

Comprueba cómo determina actualmente:

- supermercado más barato;
- productos disponibles;
- productos ausentes;
- totales;
- ahorro.

Comprueba si las comparaciones son matemáticamente correctas.

SEO

Comprueba especialmente si estamos ante una SPA dependiente de JavaScript.

Analiza:

- HTML inicial;
- SSR;
- SSG;
- prerendering;
- URLs;
- titles;
- descriptions;
- canonical;
- sitemap;
- robots;
- Schema.org.

UX

Analiza el flujo:

HOME → BUSCAR → PRODUCTO → CESTA → COMPARAR → RESULTADO.

Identifica pasos innecesarios.

RESULTADO

Entregame un informe estructurado:

1. Arquitectura actual

Explica qué tenemos realmente.

2. Lo que está bien

No debemos tocarlo sin necesidad.

3. Problemas P0

Críticos.

4. Problemas P1

Importantes.

5. Problemas P2

Mejoras.

6. Problemas P3

Futuro.

7. Deuda técnica

8. Riesgos

9. Arquitectura recomendada

Pero adaptada al proyecto existente.

10. Plan de implementación

Indica exactamente en qué orden deberíamos trabajar.

NO IMPLEMENTES TODAVÍA.

Primero quiero revisar tu diagnóstico antes de autorizar cambios.

PROMPT 2 — SEO + SSR/SSG + PÁGINAS INDEXABLES

Este lo usaría **después de recibir el informe del Prompt 1**.

Editar

Partiendo de la auditoría anterior de **Mi Mejor Cesta**, implementa la fase **P0 de SEO técnico**.

IMPORTANTE:

Antes de modificar nada, vuelve a comprobar la arquitectura real del proyecto.

No cambies de framework ni reconstruyas la aplicación salvo que exista una justificación técnica imprescindible.

Conserva todas las funciones actuales que trabajen correctamente.

PROBLEMA

Necesitamos que el contenido público relevante pueda ser correctamente:

- rastreado;
- renderizado;
- indexado;
- compartido.

No quiero depender exclusivamente del contenido generado mediante JavaScript en cliente para las páginas SEO.

OBJETIVO

Crear una arquitectura híbrida:

APLICACIÓN

Para la funcionalidad interactiva.

PÁGINAS PÚBLICAS INDEXABLES

Para productos, categorías, supermercados y comparaciones.

IMPLEMENTAR

Según permita el framework existente, utiliza:

- SSR;
- SSG;
- prerendering;

o la estrategia técnicamente apropiada.

NO introduzcas otra tecnología si la actual permite resolverlo correctamente.

URLS

Diseña una estructura preparada para:

/productos/

/producto/{slug}

/categoria/{slug}

/supermercado/{slug}

/comparar/{slug}

No generes páginas vacías o artificiales.

Cada URL indexable debe contener información útil.

METADATOS

Implementar correctamente:

- title;
- meta description;
- canonical;
- Open Graph;
- robots meta.

SITEMAP

Crear sitemap.xml dinámico o generado automáticamente.

Debe incluir únicamente páginas públicas indexables.

ROBOTS

Revisar/crear robots.txt.

No bloquear accidentalmente recursos necesarios.

SCHEMA.ORG

Preparar cuando corresponda:

- Product;
- Offer;
- AggregateOffer;
- BreadcrumbList.

Los valores deben proceder de datos reales.

Nunca generar precios ficticios.

PRODUCTO

Una página de producto debería poder mostrar:

- producto;
- marca;
- formato;
- precio mínimo;
- diferentes ofertas;
- supermercado;
- precio unitario;
- fecha de actualización.

Solamente cuando esos datos existan.

IMPORTANTE

No generes miles de páginas automáticamente únicamente para conseguir tráfico SEO.

Evita:

- thin content;
- duplicados;
- páginas prácticamente idénticas;
- combinaciones artificiales ciudad/producto.

VALIDACIÓN

Después de implementar:

1. comprueba rutas;
2. comprueba HTML inicial;
3. comprueba metadatos;
4. comprueba canonical;
5. comprueba sitemap;
6. comprueba robots;
7. comprueba JSON-LD;
8. comprueba que la SPA continúa funcionando;
9. comprueba build;
10. comprueba errores.

Al terminar, dame:

- archivos modificados;
- qué se ha implementado;

- pruebas realizadas;
- problemas encontrados;
- mejoras pendientes.

No continúes con otras fases.

PROMPT 3 — PRODUCTO, OFERTA Y BASE DE DATOS

Editar

Trabaja sobre **Mi Mejor Cesta**.

Quiero implementar correctamente la arquitectura de datos relacionada con:

PRODUCTO → SUPERMERCADO → OFERTA → PRECIO → HISTÓRICO

Antes de cambiar nada, analiza nuevamente el modelo existente.

NO destruyas datos.

NO elimines tablas o campos sin comprobar dependencias.

Si es necesario migrar información, diseña primero una migración segura.

PRINCIPIO

Un producto NO debe ser lo mismo que un precio.

Ejemplo:

PRODUCTO:

Coca-Cola Zero 2 L

OFERTAS:

Carrefour → 2,19 €

Día → 2,25 €

Alcampo → 2,15 €

MODELO CONCEPTUAL

Adapta estas entidades al sistema existente.

PRODUCTS

Debe poder representar:

- id;
- EAN;
- nombre;
- marca;
- variante;
- categoría;
- cantidad;
- unidad;
- cantidad normalizada;
- imagen;
- estado.

No todos los campos tienen que ser obligatorios.

RETAILERS

Cadena:

- Mercadona;
- Carrefour;
- Lidl;
- etc.

No confundas cadena con establecimiento físico.

STORES

Preparar arquitectura para establecimientos concretos:

- retailer;
- dirección;
- código postal;
- ciudad;
- latitud;

- longitud.

No es obligatorio activar geolocalización todavía.

OFFERS

Relacionar:

producto + supermercado/establecimiento.

Debe poder contener:

- precio;
- precio unitario;
- disponibilidad;
- promoción;
- fuente;
- fecha observada.

PRICE_HISTORY

Guardar cambios históricos.

No sustituir permanentemente precios antiguos.

REQUISITO FUNDAMENTAL

Cada precio debe poder responder a:

¿QUÉ producto?

¿EN QUÉ supermercado?

¿CUÁNTO?

¿CUÁNDO?

¿DE QUÉ fuente?

MIGRACIÓN

Si actualmente los datos están estructurados de otra manera:

1. documenta estructura actual;
2. diseña migración;
3. crea backup lógico cuando corresponda;
4. migra;

5. valida;
6. conserva compatibilidad temporal si es necesaria.

NO INVENTES

No crees precios, EAN o información inexistente.

NULL es preferible a un dato inventado.

RENDIMIENTO

Añade índices apropiados para consultas frecuentes.

Especialmente:

- EAN;
- producto;
- retailer;
- categoría;
- fechas.

AL TERMINAR

Ejecuta las pruebas necesarias.

Dame:

- modelo anterior;
- modelo nuevo;
- migraciones realizadas;
- archivos modificados;
- pruebas;
- posibles incompatibilidades.

No continúes todavía con la cesta.

Ahora vienen los prompts que puedes reutilizar tal cual después de completar esos tres. Para no convertir la respuesta en 30 páginas, estos pueden ser algo más compactos pero mantienen las restricciones importantes.

PROMPT 4 — Normalización de productos

Actúa como ingeniero de datos senior especializado en catálogos de productos.

Trabaja sobre Mi Mejor Cesta y la arquitectura Product/Offer implementada

OBJETIVO:

Conseguir que las comparaciones entre productos sean fiables.

Implementa un sistema de normalización que contemple:

- EAN cuando exista.
- Marca.
- Nombre normalizado.
- Variante.
- Cantidad.
- Unidad.
- Conversión de g $\rightarrow$ kg.
- ml $\rightarrow$ L.
- unidades.
- €/kg.
- €/L.
- €/100 g cuando tenga sentido.
- €/unidad.

Distingue obligatoriamente:

1. MISMO PRODUCTO.
2. PRODUCTO EQUIVALENTE.
3. PRODUCTO ALTERNATIVO.

Nunca determines que dos productos son idénticos únicamente porque sus n

Un EAN idéntico puede considerarse una señal fuerte de identidad.

Implementa una capa de equivalencias preparada para revisión manual.

Ejemplo:

Coca-Cola Zero 2L y Pepsi Zero 2L NO son el mismo producto.

Pueden ser alternativas.

Comprueba especialmente formatos:

500 g vs 1 kg.
750 ml vs 1 L.
packs.
2x500 g.
6 unidades.
promociones multipack.

No muestres como "más barato" un formato simplemente porque su precio ab

Utiliza precio normalizado.

Añade pruebas unitarias para conversiones y casos límite.

NO inventes equivalencias.

Al terminar:

- explica algoritmo;
- archivos modificados;
- casos de prueba;
- limitaciones.

PROMPT 5 — La cesta

Trabaja ahora exclusivamente sobre la CESTA de Mi Mejor Cesta.

No rediseñes toda la aplicación.

OBJETIVO:

Convertir la cesta en el núcleo del producto.

Implementa o mejora:

- añadir producto;
- eliminar;
- cambiar cantidad;
- vaciar cesta;
- totales;
- persistencia;
- recuperación tras recargar;
- estados vacíos;
- errores.

USUARIO ANÓNIMO:

La cesta debe sobrevivir a recargas y navegación utilizando el mecanismo

USUARIO REGISTRADO:

Si el proyecto dispone de cuentas, sincronizar la cesta con backend cuando

No rompas el sistema actual de autenticación.

Cada línea debe contemplar:

producto
cantidad
formato
preferencia de sustitución

Añade:

"Solo producto exacto"

y

"Permitir alternativas"

si la arquitectura actual permite implementarlo correctamente.

La cesta debe mostrar:

número de productos;
total estimado cuando sea posible;
productos sin precio;
productos no disponibles.

NO calcules falsos totales.

Si faltan datos, indícalo.

Prepara además arquitectura para:

- Compra semanal.
- Compra mensual.
- Cestas guardadas.

Comprueba desktop y móvil.

Añade pruebas.

Al finalizar explícame exactamente qué has cambiado.

PROMPT 6 — EL COMPARADOR

Este es uno de los más importantes de todos.

Trabaja exclusivamente sobre el COMPARADOR de Mi Mejor Cesta.

OBJETIVO:

Responder correctamente:

"¿En qué supermercado me sale más barata esta cesta?"

No quiero simplemente una tabla de precios.

Quiero un MOTOR DE DECISIÓN.

Para cada supermercado calcula:

- productos solicitados;
- productos encontrados;
- productos no encontrados;
- cobertura;
- total;

- diferencia respecto al ganador.

REGLA CRÍTICA:

NO declares ganador a un supermercado cuya cesta sea artificialmente más

Ejemplo:

Supermercado A:
20/20 productos
52 €

Supermercado B:
15/20 productos
43 €

B NO puede mostrarse simplemente como ganador.

Mostrar cobertura:

15/20 productos – 75 %

Clasifica claramente:

CESTA COMPLETA

y

CESTA INCOMPLETA.

Para cestas completas:

determina el supermercado más barato.

RESULTADO PRINCIPAL:

🏆 Mejor opción

Nombre supermercado

Total: XX €

Ahorro: XX €

Mostrar después las demás alternativas:

Supermercado B
+2,30 €

Supermercado C
+5,70 €

El ahorro debe calcularse de manera matemáticamente transparente.

Utiliza precio unitario/normalizado cuando sea necesario.

Respetar productos que el usuario haya marcado como:

"producto exacto".

No los sustituyas automáticamente.

Para productos donde permita alternativas, utiliza únicamente relaciones

No inventes sustitutos.

Añade pruebas con:

- cesta completa;
- producto ausente;
- varios productos ausentes;
- cantidades >1;
- mismo precio;
- promociones;
- formatos distintos;
- precios desactualizados;
- datos NULL.

Al terminar proporciona las fórmulas utilizadas y los tests ejecutados.

PROMPT 7 — HOME + UX/UI

Actúa ahora como Product Designer senior y especialista UX/UI.

Trabaja sobre la interfaz EXISTENTE de Mi Mejor Cesta.

NO quiero un rediseño arbitrario.

Conserva identidad, componentes y estilos que funcionen correctamente.

OBJETIVO PRINCIPAL:

Que un usuario nuevo entienda en menos de 5 segundos:

1. Qué hace Mi Mejor Cesta.
2. Qué beneficio obtiene.
3. Qué debe hacer.

Propuesta principal:

"Haz tu compra donde realmente cuesta menos."

Subtítulo:

"Crea tu cesta, compara supermercados y descubre cuánto puedes ahorrar."

CTA:

"Comparar mi cesta"

CTA secundario:

"Buscar producto"

El flujo principal debe ser:

HOME

→ BUSCAR

→ AÑADIR

→ CESTA

→ COMPARAR

→ RESULTADO

Reduce pasos innecesarios.

Añade sección:

CÓMO FUNCIONA

1. Añade tus productos.
2. Comparamos supermercados.
3. Descubre dónde ahorras.

Añade señales de confianza ÚNICAMENTE con datos reales.

Nunca inventes:

- usuarios;
- precios;
- supermercados;
- ahorro generado;
- número de productos.

Revisa:

- jerarquía visual;
- espacios;
- tipografía;
- CTAs;
- tarjetas;
- estados vacíos;
- feedback;
- loading;
- errores;
- responsive.

Prioriza AHORRO sobre cantidad de información.

No abuses de:
gradientes,
animaciones,
sombras,
iconos,
dashboards.

Busco una aplicación limpia, moderna y profesional.

Después de implementar compara:
ANTES / DESPUÉS

PROMPT 8 — CESTA INTELIGENTE

Implementa la función estratégica:

CESTA INTELIGENTE.

OBJETIVO:

Además de decir qué supermercado es más barato para comprar TODO, calcul

Mostrar dos escenarios:

OPCIÓN A

Comprar todo en un supermercado.

OPCIÓN B

Compra optimizada.

Ejemplo:

Todo en Carrefour:

58,20 €

Optimizada:

Carrefour: 30,20 €

Lidl: 19,30 €

TOTAL:

49,50 €

AHORRO:

8,70 €

IMPORTANTE:

No recomendar una fragmentación absurda.

Permitir:

Máximo supermercados:

1
2
3

Por defecto recomiendo limitar a 2.

El algoritmo debe respetar:

- productos exactos;
- alternativas autorizadas;
- disponibilidad;
- cantidades;
- precios;
- formatos;
- normalización;
- promociones válidas.

Nunca utilizar precios inexistentes.

No considerar ahorro válido si faltan productos.

Diseña el algoritmo de forma modular porque posteriormente añadiremos:

- distancia;
- coste desplazamiento;
- tiempo.

Explica matemáticamente cómo funciona.

Añade pruebas.

No implementes todavía geolocalización.

PROMPT 9 — HISTÓRICO + ALERTAS

Implementa el módulo de HISTÓRICO DE PRECIOS de Mi Mejor Cesta.

Utiliza exclusivamente datos históricos reales.

Para cada producto, cuando existan suficientes datos, mostrar:

Precio actual.
Media 30 días.
Media 90 días.
Mínimo.
Máximo.
Tendencia.

Añadir gráfico sencillo.

No mostrar estadísticas cuando no haya suficiente información.

Nunca rellenar huecos inventando precios.

Añadir:

"Avisame cuando baje de ____ €"

Si existe sistema de usuarios, guardar alertas asociadas al usuario.

Si todavía no existe infraestructura de notificaciones, prepara correcta

Diferencia:

IMPLEMENTADO

de

PREPARADO PARA IMPLEMENTAR.

Añade pruebas.

PROMPT 10 — UBICACIÓN Y AHORRO REAL

Añade una capa geográfica a Mi Mejor Cesta.

OBJETIVO:

No comparar supermercados irrelevantes para el usuario.

Permitir:

- código postal;
- ubicación autorizada;
- selección manual.

Nunca solicites ubicación sin explicar para qué se utiliza.

Relaciona retailers con establecimientos físicos cuando existan datos fi

Mostrar:

distancia aproximada.

Después prepara el concepto:

AHORRO REAL.

Ahorro real =
ahorro cesta
-
coste estimado desplazamiento adicional.

IMPORTANTE:

No presentes el coste de desplazamiento como exacto.

Debe ser configurable.

Permitir al usuario indicar:

- medio de transporte;
- coste estimado/km cuando proceda;
- máximo desplazamiento.

No inventes establecimientos ni coordenadas.

Si no existen datos geográficos fiables, muestra claramente que la función

Mantén esta lógica separada del comparador principal.

PROMPT 11 — MÓVIL + RENDIMIENTO

Realiza una auditoría y optimización de rendimiento y responsive de Mi M

NO cambies funcionalidad de negocio.

OBJETIVOS:

LCP <= 2,5 s
INP <= 200 ms
CLS <= 0,1

Trátalos como objetivos, no afirmes que se cumplen sin medirlos.

Analiza:

- bundle;
- JavaScript;
- imágenes;
- fuentes;
- requests;
- lazy loading;
- code splitting;
- caché;
- renderizados innecesarios;
- consultas;
- componentes pesados.

Optimiza sin romper funcionalidad.

MÓVIL:

Revisa desde 320 px en adelante.

Especial atención a:

- buscador;
- resultados;
- añadir;
- cesta;
- comparador;
- formularios;
- navegación.

Cuando exista cesta, considera una acción inferior persistente:

"Ver cesta · X productos"

Evita tablas horizontales enormes.

Transforma resultados en tarjetas cuando sea necesario.

Comprueba accesibilidad básica:

- teclado;
- focus;
- labels;
- contraste;
- tamaño de targets;
- aria cuando proceda.

Documenta mediciones antes/después cuando dispongas de herramientas para

No inventes métricas.

PROMPT 12 — QA FINAL

Yo **no me saltaría este**, porque es el que obliga a Claude a comprobar que después de tantos cambios no hemos roto otra parte.

Actúa como QA Engineer senior.

NO desarrolles nuevas funcionalidades.

Quiero una auditoría final completa de Mi Mejor Cesta después de las fases

PRINCIPIO:

No quiero que me digas que "todo funciona".

Quiero que intentes romper la aplicación.

TESTEA:

HOME

- navegación
- CTAs
- enlaces

BUSCADOR

- exacto
- parcial
- mayúsculas/minúsculas
- acentos
- errores
- sin resultados
- caracteres especiales

PRODUCTOS

- ficha
- precios
- unidades
- imágenes
- productos sin EAN
- productos incompletos

CESTA

- añadir
- eliminar
- cantidades
- recargar
- cerrar/volver
- vaciar
- persistencia

COMPARADOR

- cesta completa
- incompleta
- empate
- precios NULL
- cantidades
- producto ausente
- alternativas

CESTA INTELIGENTE

- 1 supermercado
- 2 supermercados
- 3 supermercados
- productos no disponibles
- ahorro cero

ChatGPT puede cometer errores. Considera verificar la información importante.